AI 出版生产平台，先从 `audiobook` 起步，但底层从第一天就按可扩展到 `漫画` / `video` / `SBS content` 来设计。

我建议的总方向是：

`Studio + Publishing Platform` 两层架构

- `Studio`： 内部生产后台，给编辑/操作员/审核人员用
- `Publishing Platform`： 对外发布、展示、交付的平台

关键原则只有一句：

先做“统一内容资产平台”，再做不同格式的生产器。

如果一开始 audiobook、漫画、video、SBS 各做一套，后面一定会散。

一、总体架构方向

我建议：

前端

- 内部生产后台： `Next.js` （基于 React）
- 对外发布站： `Next.js`

后端

- 主 API： `FastAPI`
- 异步任务： `Celery` / `Arq` / `RQ`
- 数据库： `PostgreSQL`
- 队列/缓存： `Redis`
- 对象存储： `S3` / `MinIO` / `OSS` / `R2`
- 媒体处理： `FFmpeg`
- AI 适配层：独立 `AI Gateway`
- Agent/Tool 层：独立 `Agent Runtime + MCP Tool Gateway`

为什么这样配：

- `FastAPI` 很适合 AI、媒体、异步任务、工具编排
- `Next.js` 适合做内部后台，也适合以后做公开出版站
- `Postgres + Redis + Object Storage` 足够支撑前中期
- 以后扩展不会推翻重来

二、最重要的设计：统一内容模型

这个平台不要按“音频平台 / 漫画平台 / 视频平台”建模。
要按 **作品**、**章节**、**单元**、**资产**、**版本**、**发布** 来建模。

我建议核心抽象是：

- `Work`
- `Edition`
- `Chapter`
- `Unit`
- `Asset`
- `AssetVersion`
- `WorkflowJob`
- `Review`
- `Release`

其中最关键的是：

1. `Work`

代表一个 IP / 一本书 / 一个作品

例子：

- 一本小说
- 一个原创故事宇宙
- 一个角色系列

2. `Edition`

代表某种出版形态

例子：

- audiobook edition
- comic edition
- video edition
- SBS edition

这样同一个作品可以衍生多个版本，而不是复制一份项目。

3. `Unit`

这是最关键的统一抽象。

不同格式的最小生产单元都统一叫 `Unit`，只是在类型上不同：

- audiobook： `segment`
- comic： `panel`
- video： `shot`
- SBS： `stereo_clip` 或 `scene`

这样前后台流程能保持连贯。

4. `Asset`

所有实际文件统一归一到 Asset：

- 原始文本
- 清洗文本
- 音频
- 图片
- 视频
- 封面
- 字幕
- 波形
- 缩略图
- stereo metadata

5. `AssetVersion`

所有生成、编辑、重做都不覆盖，全部有版本。

这点必须从第一天定死。
因为以后视频、漫画、SBS 都会频繁返工。

三、推荐的整体系统分层

我建议按下面 6 层来做：

```
[Studio Web / Public Web]
          |
       [API Layer]
          |
[Domain Services / Workflow Services]
          |
[AI Gateway] [Media Pipeline] [Agent Runtime]
          |
 [Postgres] [Redis] [Object Storage]
```

具体解释：

1. Web 层

分成两个前端：

`studio-web`

给内部人员用：

- 导入
- 编辑
- 批量生成
- 校对
- 审核
- 发布

`public-web`

给读者/用户用：

- 浏览作品
- 试听试看
- 购买或访问
- 阅读漫画
- 看视频
- 播放 SBS 内容

---

2. API 层

用 `FastAPI` 做统一 API。

职责：

- 用户认证
- 项目/作品管理
- 章节与单元管理
- 任务创建
- 版本读取
- 审核与发布
- 对外公开内容接口

建议：

- 内部后台 API 和发布 API 逻辑上分 namespace
- 但先放一个 FastAPI 项目里，保持 **模块化单体**

我不建议一开始就微服务化。

---

3. Domain Services

这是业务核心，不要让前端或 Agent 直接碰数据库。

拆成这些模块最合理：

- `identity`
- `catalog`
- `content_ingest`
- `workflow`
- `asset_management`
- `review_and_approval`
- `publishing`
- `billing_cost`
- `provider_registry`

---

4. AI Gateway

这是平台未来是否连贯的关键。

不要让业务代码直接调用：

- ElevenLabs
- OpenAI
- Seedream
- Seedance
- Veo
- 任何模型 API

统一走 `AI Gateway` 。

它负责：

- provider adapter
- 参数标准化
- prompt 模板版本
- request id 记录
- credits / cost 统计
- fallback
- provider 切换

举例：

同一个“生成 audiobook segment”的业务请求，内部可能映射到：

- ElevenLabs
- OpenAI TTS
- 以后别的 TTS

业务层不应该知道差异。

---

5. Media Pipeline

这是处理文件的层，不是 AI 层。

职责：

- 切分文本
- 音频拼接
- 波形生成
- loudness 检查
- ASR 回听
- 图片合成
- 视频转码
- 字幕烧录
- SBS 资源封装

工具主要是：

- `FFmpeg`
- `ImageMagick`
- 自定义 Python 工具
- 以后可加 OpenCV / moviepy / pydub

---

6. Agent Runtime + MCP Tool Gateway

你提到未来要整合各种 agent、MCP、LLM、剪辑工具。

我建议：

MCP 不要当主后端。
MCP 应该是“工具接入层”，不是你的核心业务 API。

正确位置是：

- 平台核心业务： `FastAPI + Domain Services`
- Agent：调用平台服务 + 调用 MCP tools
- MCP：把外部工具标准化给 agent 用

也就是说：

平台本身负责

- 数据
- 权限
- 工作流
- 审核
- 版本
- 发布

Agent 负责

- 文本清洗
- 自动拆章拆段
- 发音建议
- storyboard 生成
- shot list 生成
- 漫画对白提炼
- QC 建议
- 标题/标签/简介生成

MCP 负责

- 让 agent 能稳定调用工具
- 比如：
  - `ffmpeg`
  - `subtitle`
  - `image editor`
  - `speech provider`
  - `video provider`
  - `stereo packaging tool`

一句话：

MCP 是工具层，不是业务层。

四、前端怎么设计更连贯

我建议前端不要只做一个“后台页面集合”，而是做成两类应用。

1. 内部生产前端： `Next.js`

比纯 React SPA 更适合你这个项目。

原因：

- 后面你一定会有内部后台 + 对外页面
- `Next.js` 更适合做多入口
- 路由、鉴权、SSR、文件上传、服务端渲染内容页都方便
- 以后 public publishing 也能复用

建议前端栈：

- `Next.js`
- `TypeScript`
- `TanStack Query`
- `Zustand`
- `Tailwind`
- `shadcn/ui`
- `wavesurfer.js` 用于音频波形
- 后期漫画可加 `Konva` / `Fabric.js`
- 后期视频可加 timeline 组件

2. 对外发布前端： 也用 `Next.js`

这样你的 Studio 和 Public 两端技术统一。

五、后端怎么设计更稳

1. 先做模块化单体，不做微服务

这是我最明确的建议。

为什么：

- 你现在不是高并发消费品平台
- 你现在的核心难点是工作流、媒体、AI、版本，不是服务拆分
- 微服务会把复杂度提前 10 倍

所以先做：

- 一个 `FastAPI` 主服务
- 一个 `worker` 服务
- 一个 `ai-gateway` 模块
- 一个 `media` 模块

逻辑拆清楚，进程可以先少。

2. 用异步任务做所有重活

任何长任务都不要在请求里跑：

- TTS 批量生成
- ASR 质检
- 波形生成
- 音频拼接

- 视频渲染
- 图片批量生成
- SBS 打包

这些全部扔到队列里。

建议：

- 早期：`Redis + Celery/Arq`
- 后期流程更复杂时：再考虑 `Temporal`

我建议 **现在不要急着上 Temporal**，除非你团队已经熟。

六、从 audiobook 起步，未来如何平滑扩展

这部分最重要。

Phase 1：Audiobook

先做这些域对象：

- Work
- Chapter
- Segment
- AudioAsset
- AudioTake
- ReviewIssue
- ExportTask

核心流程：

- 导入文本
- 拆章拆段

- TTS
- ASR 质检
- 校对
- 审核
- 导出

Phase 2：Comic

不要新建“漫画系统”，而是在同一平台里新增：

- Edition type = `comic`
- Unit type = `panel`
- Asset type = `image`，`layout`，`dialogue`

复用原有：

- Work
- Chapter
- Review
- Version
- Publish

新增：

- 角色参考图
- 分镜脚本
- 面板布局器

Phase 3：Video

新增：

- Edition type = `video`
- Unit type = `shot`
- Asset type = `keyframe`，`video_clip`，`subtitle`

复用：

- 工作流
- 审核
- 发布
- 版本
- provider gateway

Phase 4：SBS Content

新增：

- Edition type = `sbs`
- Unit type = `stereo_scene`
- Asset type = `left_right_frame`，`depth`，`stereo_metadata`，`device_profile`

复用：

- 作品
- 章节
- 任务系统
- 审核系统
- 发布系统

也就是说：

不是每加一种内容，就加一个系统。
而是每加一种内容，只加：

- 新的 edition type
- 新的 unit type
- 新的 renderer / generator
- 新的 review rules

这样才连贯。

七、数据库层面的大方向

核心表建议一开始就按长线设计：

- `users`
- `organizations`
- `projects`
- `works`
- `editions`
- `chapters`
- `units`
- `assets`
- `asset_versions`
- `jobs`
- `reviews`
- `approvals`
- `releases`
- `provider_calls`
- `cost_records`

其中 `units` 不要写死成 `segments` 。
可以有：

- `unit_type = segment`
- `unit_type = panel`
- `unit_type = shot`
- `unit_type = stereo_scene`

这就是未来连贯性的基础。

八、建议的目录结构

我建议用一个 monorepo，但不要搞太重。

```
platform/
  apps/
    studio-web/        # Next.js 内部生产后台
    public-web/        # Next.js 对外发布站
  services/
    api/               # FastAPI 主 API
    worker/            # Celery/Arq worker
  libs/
    domain/            # 核心业务模型
    ai_gateway/        # provider adapters
    media_pipeline/    # ffmpeg / transcode / qc
    agent_runtime/     # agent orchestration
    mcp_tools/         # MCP tool wrappers
  infra/
    docker/
    migrations/
    scripts/
  docs/
```

如果你嫌 monorepo 太重，也可以先这样：

```
platform/
  web/
  api/
  worker/
  docs/
```

九、Agent 和 LLM 在平台里的正确位置

我建议从第一天就把 agent 当成“协作模块”，不是“主逻辑”。

适合 agent 做的事：

- 自动拆章拆段

- 朗读稿预处理
- pronunciation rule 建议
- 漫画 4 格分镜提炼
- video shot list 提炼
- 质检建议
- metadata/summary/tagline 生成
- SEO / 出版说明生成

不适合 agent 直接做的事：

- 直接改数据库核心状态
- 绕过审核直接发布
- 绕过业务 API 调工具
- 直接持久化生产数据

做法应该是：

Agent 输出建议 -> 人工确认或 Workflow 接收 -> 走平台 API 持久化

十、对外发布平台也要从一开始留口

虽然现在先做 audiobook，但以后是出版平台，所以 public side 要预留这些域：

- 作品页
- Edition 页
- 章节页
- 试看试听
- 购买/订阅
- 用户收藏
- 进度记录
- 搜索与标签
- 推荐位

但这些先不必实现完整。
只要数据模型里留有：

- work
- edition
- release
- publication_status
- public_metadata

后面扩展就顺。

十一、我最推荐的起步组合

如果你现在就要定方向，我建议是：

前端

- Next.js + TypeScript

后端

- FastAPI

基础设施

- PostgreSQL
- Redis
- S3-compatible storage

- `FFmpeg`

异步

- `Celery` 或 `Arq`

AI

- `AI Gateway` 统一封装
- 第一阶段接：
  - `ElevenLabs / OpenAI TTS`
  - `Whisper/ASR`
- 后续接：
  - `Seedream`
  - `Seedance`
  - `Veo`
  - SBS 处理工具

架构风格

- **模块化单体 + 异步 worker**
- **不要一开始微服务化**

十二、一句话总结

最稳的路线是：

先做一个以 `Work / Edition / Unit / Asset / Workflow / Release` 为核心的统一出版平台，前端用 `Next.js`，后端用 `FastAPI`，从 audiobook 的 `segment` 工作流起步，再逐步扩展到 comic、video、SBS。

这样未来加任何格式，都不是重做平台，只是新增一种 `edition + unit + generator + review rule`。

如果你要，我下一步可以直接继续两种之一：

1. 给你画出这套平台的 **系统架构图 + 模块图**
2. 直接写一版 **第一期 audiobook 平台的数据库与 API 草案**